

dataset/kuaiREC/kuaiREC_process.py (Q&A)

Q: What is the purpose of dataset/kuaiREC/kuaiREC_process.py? **A:** This script pre-processes raw KuaiREC CSV files into a single pickled dataset (kuaiREC_data.pkl) plus auxiliary artifacts used by some baselines (e.g. duration buckets and per-bucket play-time quantiles for D2Q). It is intended to be run once before training any models.

Q: Is this a library module or an executable script? **A:** It is written as a pure script: it executes processing steps at import time (no `main()` / `if __name__ == '__main__':` guard), so it should be run directly (and not imported) in a controlled working directory that contains the expected `row_data/` inputs.

Q: What raw files does it expect? **A:** It reads multiple CSVs under `row_data/` (e.g. `big_matrix.csv`, `user_features.csv`, `item_daily_features.csv`, `item_categories.csv`, and a caption/category file), merges them by `user_id` and `video_id`, fills missing values, and filters out invalid rows (e.g. ads and non-positive durations).

Q: What label and core continuous feature does it create? **A:** It constructs `play_time` as a normalized version of `play_duration`, and `duration` as a normalized/clipped version of `video_duration`. It also caps extreme `play_time` values relative to `duration` (a heuristic to remove outliers).

Q: How are sparse and sequence features encoded? **A:** For each sparse feature column, it builds a vocabulary mapping unique tokens to integer IDs. For the sequence feature `feat`, it tokenizes by commas, truncates/pads to length 10, and writes a corresponding `featmask` to indicate valid positions; both are recorded in the output `description` schema.

Q: What additional artifacts are created for D2Q? **A:** It discretizes duration into `n_bins=50` buckets using quantiles (`pd.qcut`), writes bucket ranges to `d2q_duration_bucket_ranges.csv`, and writes per-bucket `play_time` quantiles to `d2q_duration_bucket_playtime_quantiles.csv`.

Q: What does the final output pickle contain? **A:** The pickle contains a dict with keys `train` and `test` holding DataFrames, plus `description` containing a list of feature descriptors (`name`, `size`, `type`) used by the dataloader and models.

Q: Code example: the final saved object shape (schematic)? **A:**

```
data = {
    "train": df_train,
    "test": df_test,
    "description": desc,
}
with open("./kuaiREC_data.pkl", "wb+") as f:
```

```
pickle.dump(data, f)
```

Q: Full source code? A:

```
import pandas as pd
import numpy as np
import pickle

with open("row_data/kuaiREC_caption_category.csv", "r", encoding="utf-8", errors="ignore"
) as f:
    lines = f.readlines()

with open("row_data/cleaned_kuaiREC_caption_category.csv", "w", encoding="utf-8") as f:
    f.writelines(lines)

df5 = pd.read_csv("row_data/cleaned_kuaiREC_caption_category.csv", encoding="utf-8")
df5["video_id"].astype(int)
df5.to_csv("row_data/cleaned_kuaiREC_caption_category.csv", index=False, encoding="utf-8"
)

#join feature files to one file
df1 = pd.read_csv("row_data/big_matrix.csv")
df2 = pd.read_csv("row_data/user_features.csv")
df3 = pd.read_csv("row_data/item_daily_features.csv").loc[:, ['video_id', 'video_type', '
music_id', 'video_tag_id']]
df3 = df3.drop_duplicates(subset=['video_id'])
df4 = pd.read_csv("row_data/item_categories.csv")
df5 = pd.read_csv("row_data/cleaned_kuaiREC_caption_category.csv", encoding="utf-8").loc
[:, ['video_id', 'first_level_category_id', 'second_level_category_id', '
third_level_category_id']]

df_merged_1_2 = pd.merge(df1, df2, on="user_id", how="left")
df_merged_1_2_3 = pd.merge(df_merged_1_2, df3, on="video_id", how="left")
df_merged_1_2_3_4 = pd.merge(df_merged_1_2_3, df4, on="video_id", how="left")
df_final = pd.merge(df_merged_1_2_3_4, df5, on="video_id", how="left")

df = df_final.fillna('12345')

#filter AD video
df = df[df['video_type'] != 'AD']
df = df[df['play_duration'] > 0]
df = df[df['video_duration'] > 0]

df = df.sample(frac=0.1, random_state=312).reset_index(drop=True)

#delete unnecessary features
df = df.drop(columns=['video_type', 'time', 'date', 'watch_ratio', 'follow_user_num', '
fans_user_num', 'friend_user_num', 'register_days'])

# indentify different types of feature
label="play_duration"
```

```

sequence_feature="feat"
sparse_feature_list=['feat','user_id','video_id','music_id','video_tag_id','
    user_active_degree','is_lowactive_period','is_live_streamer','is_video_author','
    follow_user_num_range','fans_user_num_range','friend_user_num_range','
    register_days_range','onehot_feat0','onehot_feat1','onehot_feat2','onehot_feat3','
    onehot_feat4','onehot_feat5','onehot_feat6','onehot_feat7','onehot_feat8','
    onehot_feat9','onehot_feat10','onehot_feat11','onehot_feat12','onehot_feat13','
    onehot_feat14','onehot_feat15','onehot_feat16','onehot_feat17','
    first_level_category_id','second_level_category_id','third_level_category_id']
dense_feature_list=['video_duration','timestamp']

# create feature desc and process play_time_ms/duration_ms
desc=[('play_time', -1, 'label'), ('duration', -1, 'ctn')]
df['play_time'] = df['play_duration'] / df['play_duration'].max()
df['duration'] = df['video_duration'].clip(upper=df['play_duration'].max()) / df['
    play_duration'].max()
df = df[df['play_time'] < df['duration']*10]
df['play_time'] = df.apply(lambda row:row['play_time'] if row['play_time'] < row['
    duration']*10 else row['duration']*10,axis=1)
df['play_time'] = df['play_time'].astype(float)
df['duration'] = df['duration'].astype(float)

# preprocess sparse features:
for sparse_feature_name in sparse_feature_list:
    sparse_feature_set= set()
    sparse_feature_col = []
    for index, row in df.iterrows():
        if (sparse_feature_name == sequence_feature):
            sparse_feature = [str(i) for i in row[sparse_feature_name].split(",")]
            sparse_feature_set.update(sparse_feature)
            sparse_feature_col.append(sparse_feature)
        else:
            sparse_feature = str(row[sparse_feature_name])
            sparse_feature_set.add(sparse_feature)
            sparse_feature_col.append(sparse_feature)

# generate the vocabulary of search tokens and items
sparse_feature_voc = {word: idx for idx, word in enumerate(sparse_feature_set)}
print("sparse_feature_{}_size:{}".format(sparse_feature_name,len(sparse_feature_voc)
    ))

# reset query and item index with new vocabulary
for idx, ori_feature in enumerate(sparse_feature_col):
    if (sparse_feature_name == sequence_feature):
        sparse_feature_col[idx] = [sparse_feature_voc[word] for word in ori_feature]
    else:
        sparse_feature_col[idx] = sparse_feature_voc[ori_feature]

if(sparse_feature_name == sequence_feature):
    sparse_mask_col= []
    for idx, ori_feature in enumerate(sparse_feature_col):
        feature_len = len(ori_feature[:10])
        new_feature = ori_feature[:10] + [0]*(10-feature_len)
        feature_mask = [1.0]*feature_len + [0.0]*(10-feature_len)

```

```

        sparse_feature_col[idx] = new_feature
        sparse_mask_col.append(feature_mask)
    df['featmask'] = sparse_mask_col
    df[sparse_feature_name] = sparse_feature_col

    # delete features that only has one value and adding desc
    if len(sparse_feature_voc) == 1:
        df = df.drop(sparse_feature_name, axis=1)
        print("remove the {} feature!!!!".format(sparse_feature_name))
    else:
        if (sparse_feature_name == sequence_feature):
            desc.append((sparse_feature_name, len(sparse_feature_voc), 'seq'))
            desc.append(('featmask', -1, 'seqm'))
        else:
            desc.append((sparse_feature_name, len(sparse_feature_voc), 'spr'))

# generate precomputed duration buckets for D2Q
n_bins = 50
df['duration_bucket'], bins = pd.qcut(df['duration'], q=n_bins, labels=False, retbins=True,
    duplicates='drop' )
bucket_ranges = pd.DataFrame({'bucket_index': range(len(bins)-1), 'min_duration': bins
    [:-1], 'max_duration': bins[1:]})
bucket_ranges.to_csv('d2q_duration_bucket_ranges.csv', index=False)
desc.append(('duration_bucket', len(bins) - 1, 'spr'))

# split train, test set
df = df.sample(frac=1, random_state=1234).reset_index(drop=True)
df_train = df[:int(0.8*len(df))]
df_test = df[int(0.8*len(df)):]

# generate precomputed quantiles in each duration bucket for D2Q
quantile_num= 100
quantiles = np.linspace(0, 1, quantile_num+1)
quantile_df = (df_train.groupby('duration_bucket')['play_time'].quantile(quantiles).
    reset_index().rename(columns={'level_1': 'quantile'}))
quantile_pivot = quantile_df.pivot(index='duration_bucket', columns='quantile', values='
    play_time')
quantile_pivot.to_csv('d2q_duration_bucket_playtime_quantiles.csv')

# save to pickle
data = {
    "train": df_train,
    "test": df_test,
    "description": desc
}
with open('./kuairec_data.pkl', 'wb+') as f:
    pickle.dump(data, f)

```